PRESENTER RUNBOOK / FACULTY DEMO

From shop to security report

A guided 20-minute story showing the working target, selected weaknesses, the assessment button, and the evidence-backed outputs.

Storyline: Alice is shopping at Northstar Supply. A routine store visit raises questions about privacy and trust. The team uses the application itself, then a local assessment run, to inspect those questions and produce a report.

The goal of the presentation

Show a practical end-to-end loop: use the storefront as a customer, demonstrate a few bounded weaknesses and their secure comparisons, run the assessment, then explain the report and the limits of its claims. The story is intentionally evidence-first: a scanner observation is a candidate, not an automatic verdict.

Time	Presenter action	Audience takeaway
0-2 min	Set scope and introduce Northstar Supply.	This is a local practice target with normal shopping flows.
2-7 min	Use the store and show selected paired weaknesses.	The weak behavior is embedded in real routes, not hard-coded lesson cards.
7-12 min	Run the assessment and inspect stage progress/results.	One button orchestrates multiple modules and persists a traceable run.
12-17 min	Open report, findings, paths, and remediation.	Outputs include evidence and uncertainty; a candidate is not a confirmed issue.
17-20 min	Explain retest, limitations, and close.	Fixes can be checked again while preserving reviewer control.

Before the presentation

- Use a Mac/Linux terminal or a terminal with Docker Compose, Python 3.11+ (project uses its local virtual environment), and a browser.
- Open the repository root: /Users/adithyaprasad1512/Programming/web-security-framework.

- Confirm port 5001 and port 8010 are free. Keep the demo on the same machine.
- Use a clean browser profile or sign out of Northstar so you can show sign-in explicitly.
- Keep the assessment terminal visible during the run; it prints the stage names and status.
- Option A, show the existing run first: assessment #15 completed 10 stages, with 21 candidates and 7 candidate paths. This avoids waiting on a fresh run.

Start the two services

Terminal 1 - start Northstar Supply:

```
cd /Users/adithyaprasad1512/Programming/web-security-framework/target-app docker compose up --build -d
```

Then open http://localhost:5001/. Check that the featured product and the product list load. This is a database-backed store. Checkout only adds local sample orders; it does not charge a card.

Terminal 2 - start the assessment desk:

```
cd /Users/adithyaprasad1512/Programming/web-security-framework/framework source .venv/bin/activate python dashboard.py
```

Then open http://localhost:8010/. The desk shows the newest saved assessment. For the known example, open http://localhost:8010/?assessment_id=15.

> **If the demo machine cannot use Docker, start the target using its existing local setup and dependencies. Do not expose port 5001 outside the computer.**

A short customer storyline

"Alice wants a keyboard. She expects her account, orders, reviews, and documents to stay private. We will use the shop normally, then check whether a few ordinary routes honor those expectations."

1. Tour the storefront (1 minute)

- Point out the featured product, dynamic catalog, search, account, reviews, and order support.
- Mention that the editorial layout is generated from the actual store data and that the review list is empty until someone posts.
- Do not begin with source code. Establish that the application is useful before discussing its weaknesses.

2. Show account and order behavior (1-2 minutes)

- Sign in as alice / alice123. The store shows Alice's own orders. The administrator credential for later is admin / admin123.
- Explain that orders are local fixtures and checkout is a demonstration flow, with no payment provider.
- For object ownership, show the ordinary order link and explain the comparison: the insecure API does not verify the owner; the paired /api/orders/<id> route does. Let the assessment perform the identity comparison instead of

guessing another user's order ID.

3. Show two bounded input/output examples (2-3 minutes)

- Reflected output: open `/products/preview?q=%3Cem%3Edemo%3C%2Fem%3E`. Compare `/products/preview/secure` with the same query. Use the inert emphasis tag only; no script runs.
- Stored output: post a harmless review such as `Desk note: <em>nice finish</em>`. Compare the page and /reviews/secure. Delete the manual review from admin operations when done. The scanner itself creates a unique harmless marker and removes it through the protected moderator route.
- If time allows, show DOM output with a URL fragment on /products/dom-preview#%3Cem%3Edemo%3C%2Fem%3E, then compare /products/dom-preview/secure. Point out innerHTML versus textContent.

Show the assessment as the main event

4. Run a fresh assessment (allow several minutes)

- Return to the assessment desk at http://localhost:8010/.
- Leave the default target http://localhost:5001 in the Target URL field.
- Click Run assessment once. Keep the terminal visible while it works; do not start a second run.
- The local-target guard rejects external hosts, non-HTTP schemes, other ports, and non-root target paths.
- The page will refresh when the run completes. Note the assessment number and whether the status is COMPLETED or PARTIAL.

What the audience should see

Dashboard area	What to narrate
Workflow	Each named stage writes SUCCEEDED, FAILED, or SKIPPED plus details. Reconnaissance and role checks come before analysis and reporting.
Findings and remediation	Each row has category, lifecycle state, project risk, and a suggested defensive action. Do not call candidates confirmed vulnerabilities.
Retest	After a code fix, this records new authorization and ownership evidence against the original assessment. It does not auto-close findings.
Attack-path analysis	Paths connect evidence-backed candidates as hypotheses. The workflow does not claim these chains were exploited end to end.
History	Past runs remain available for comparison. Select a prior assessment by its number.
HTML and JSON report links	HTML is for people. JSON is for structured review and downstream processing.

5. Open the report (2-3 minutes)

- Click Open HTML report and point out summary counts, stage outcomes, evidence, risk, paths, and limitations.
- Open the JSON report only if a teammate wants to inspect the structured representation.

- Explain the distinction between candidate, verified, remediated, and reopened statuses. Automation records observations; people decide lifecycle changes.
- Explain the risk score is the project's preliminary method, not CVSS.

6. Explain optional ZAP import (1 minute)

- The framework does not launch OWASP ZAP. It imports a supplied, scope-matched ZAP JSON report and performs safe live-response checks for relevant alerts.
- The repo includes a baseline artifact that may be old. For a current ZAP demonstration, create a fresh report for localhost:5001 and pass it to the CLI with --zap-report /path/to/report.json.
- If no artifact exists, the ZAP-related stages are skipped and the overall assessment can be PARTIAL. This is expected behavior, not necessarily an application failure.

7. Show a remediation retest (optional, only after a fix)

Apply a real code fix to a paired secure route, rebuild/restart the target, select the assessment, and click Record remediation retest. The retest checks admin-role access and object ownership. It appends evidence and refreshes the report, but preserves finding status so a reviewer can decide whether the fix is sufficient.

> **Do not present the retest button as an automatic remediation engine. The project provides suggested remediations and a way to collect new evidence; a developer still implements the code change.**

Safe local examples for the live demo

Topic	Local URL or action	Expected explanation
SQL search	/api/products/search?q=%27%20OR%201%3D1%20--%20	A read-only boolean probe used by the assessment. Compare with /api/products/search/secure. Do not point it at another service.
Admin access	Open /admin/lab/users in a clean anonymous session, then compare protected admin routes.	The lab route intentionally exposes a read-only user listing; role-aware checks compare anonymous, Alice, and admin.
Path traversal	/api/files/download?name=../private/lab-note.txt	The vulnerable route can read only within the lab fixture root. Compare /api/files/download/secure; it constrains access to the public fixture.
CSRF	Compare /account/preferences with /account/preferences/secure.	The scanner inspects token presence and does not submit a state change.
Security headers	Review response observations in the report.	Header observations are candidates to review against deployment needs.

Wrap-up words

"We started with a real application flow, collected observations under a local scope, and turned them into traceable candidate findings and reports. The results show where to investigate and how to retest after a fix. The system keeps evidence and uncertainty visible instead of presenting every scanner signal as a confirmed vulnerability."

Troubleshooting

Symptom	What to check
Store page will not open	Docker is running; run docker compose up --build -d from target-app; inspect docker compose logs.
Dashboard cannot reach target	Open http://localhost:5001/ first. Confirm target is healthy and port 5001 is bound locally.
Dashboard port in use	Stop the previous dashboard process or choose a free port only if the dashboard binding and links are intentionally updated.
Run ends PARTIAL	Read the exact failed or skipped stage details. If ZAP artifact was absent or stale, provide a fresh scope-matched artifact; do not hide a partial status.
Reports not found	Use the assessment ID shown by the dashboard/terminal and look under the repository reports/ directory.
Database state looks different	Target fixtures persist in target-app/instance/security_lab.db. New reviews/orders can change demo state. Use a fresh demo environment when a clean presentation is important.

Stop after the presentation

Stop the dashboard with Ctrl+C in Terminal 2. Stop the target with `cd target-app && docker compose down` (or run docker compose down from the target-app directory). This stops the container but preserves the local database volume unless you explicitly remove it.

Quick checklist

- Both sites are local and reachable.
- The target remains loopback-only.
- You have an assessment ID and report links.
- You explain candidate status, project-specific risk, optional ZAP input, and retest limits.
- You end with the workflow: observe -> record evidence -> review -> remediate -> retest.